SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: MLA03
COURSE NAME: Reinforcement learning for Environment and Agent

---

COURSE OUTCOME COVERED

CO3: *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)*

Assessment Tool Weightage: 30%

Dr G. A. SENTHIL

QUESTIONS: 10 **Total Marks: 50**

Annexure A

CODING CHALLENGE **Duration: 2hrs**

Code of Conduct:

I Vignesh S / 192325002 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Vignesh S

REINFORCEMENT LEARNING CODING CHALLENGE

General Challenge Information

Overview: This coding challenge assesses the practical implementation of core reinforcement-learning concepts. It contains ten independent tasks, each targeting one fundamental RL idea, from the agent-environment loop through to on-policy and off-policy control. For every task the candidate must implement a correct, runnable solution and reproduce the stated output.

Eligibility: Open to all students enrolled in MLA03 (Reinforcement Learning for Environment and Agent) who have completed the CO1 and CO2 assessments. Each candidate submits individually; group submissions are not accepted.

Challenge Rules: All code must be the candidate's original work. Each task is solved independently and must run without manual editing and reproduce the reported output. Standard scientific libraries are permitted, but external reinforcement-learning frameworks that solve the task directly are not. Every submission must include the source code together with its captured console output. Random seeds are fixed so results are reproducible.

Programming Languages: Python 3 is the required language. Solutions use only NumPy and the Python standard library, so no specialised RL toolkit is needed and every script runs on a standard Python installation.

Environment Setup: Install Python 3.8 or later and NumPy (pip install numpy); Matplotlib is optional for visualisation. Each solution is a self-contained script with a fixed random seed and is executed with the command python <n>.py, printing its results to the console.

Challenge 1: Agent-Environment Interaction and Cumulative Reward

Problem Statement: Implement a minimal agent-environment interaction loop for a six-state corridor in which the agent must reach a goal. Demonstrate the roles of states, actions and rewards, and show how the cumulative reward, or return, decides which policy the agent prefers.

Objectives: Build the environment step function, run an episode under a given policy, accumulate reward across the episode, and compare two policies by their total return.

Input Format: No external input. The corridor length, goal position and per-step and goal rewards are fixed constants inside the program.

Output Format: A step-by-step interaction trace of (state, action, reward) for the optimal policy, followed by the cumulative reward of each policy compared.

Constraints: States 0 to 5; actions are left and right; reward of minus one per step and plus ten at the goal; each episode is capped at thirty steps.

Evaluation Criteria: Correct environment dynamics, correct return computation, and a clear demonstration that the policy with the higher cumulative reward is preferred.

Reference Solution (Python):

```
# Challenge 1: Agent-Environment interaction & cumulative reward
# A 1D corridor: states 0..5, goal at 5. Actions: 0=left, 1=right.
# Reward -1 per step, +10 on reaching the goal. We compare two policies
# to show how cumulative reward (the return) drives which policy is better.
import numpy as np
GOAL = 5

def step(s, a):
    s2 = min(GOAL, s + 1) if a == 1 else max(0, s - 1)
    reward = 10 if s2 == GOAL else -1
    done = s2 == GOAL
    return s2, reward, done

def run(policy, name, trace=False):
    s, total, steps = 0, 0, 0
    if trace: print(f"\n{name} policy trace:")
    while True:
        a = policy(s)
        s2, r, done = step(s, a)
        total += r; steps += 1
```

```
        if trace: print(f"  state {s} --action {'right' if a else 'left'}--> state {s2} | reward {r:
+d}")
        s = s2
        if done or steps > 30: break
    print(f"{name:<16} cumulative reward (return) = {total:>4} in {steps} steps")
    return total

np.random.seed(0)
run(lambda s: 1, "Always-right", trace=True)          # optimal
run(lambda s: np.random.randint(2), "Random")         # weaker
print("\nThe agent prefers the policy with the higher cumulative reward.")
```

Sample Output:

```
Always-right policy trace:
  state 0 --action right--> state 1 | reward -1
  state 1 --action right--> state 2 | reward -1
  state 2 --action right--> state 3 | reward -1
  state 3 --action right--> state 4 | reward -1
  state 4 --action right--> state 5 | reward +10
Always-right     cumulative reward (return) =    6 in 5 steps
Random           cumulative reward (return) =    3 in 8 steps

The agent prefers the policy with the higher cumulative reward.
```

Challenge 2: Modelling a Decision Problem as a Markov Decision Process

Problem Statement: Model a simple weather-based decision problem as a Markov Decision Process and evaluate a fixed policy. Define the action space, transition probabilities, reward function and discount factor, then compute the value of each state.

Objectives: Represent the transition probabilities, reward function and discount factor; assemble the policy-induced transition matrix and reward vector; and solve the linear Bellman system for the state values.

Input Format: No external input; the MDP, comprising states, actions, transition probabilities, rewards and the discount factor, is defined in code.

Output Format: The action space, discount factor, policy-induced transition matrix, reward vector and the computed state values V(s).

Constraints: Two states and two actions; discount factor gamma equal to 0.9; each transition-probability row sums to one.

Evaluation Criteria: A correct MDP formulation and a correct closed-form policy evaluation using V = (I - gamma P) inverse times R.

Reference Solution (Python):

```
# Challenge 2: Model a decision problem as a Markov Decision Process (MDP).
# Components: state space S, action space A, transition probability P,
# reward function R, discount factor gamma. We evaluate a fixed policy by
# solving the linear system V = R + gamma * P @ V  ->  V = (I - gamma P)^-1 R.
import numpy as np
np.random.seed(0)

S = ["Sunny", "Rainy"]          # state space
A = ["Walk", "Shop"]            # action space
gamma = 0.9                     # discount factor

# Transition probabilities P[s][a] -> distribution over next states
P = {
 "Sunny": {"Walk": [0.8, 0.2], "Shop": [0.6, 0.4]},
```

```
 "Rainy": {"Walk": [0.4, 0.6], "Shop": [0.5, 0.5]},
}
# Reward function R[s][a]
R = {
 "Sunny": {"Walk": 5, "Shop": 2},
 "Rainy": {"Walk": -2, "Shop": 3},
}
policy = {"Sunny": "Walk", "Rainy": "Shop"}   # fixed policy to evaluate

Pmat = np.array([P[s][policy[s]] for s in S])
Rvec = np.array([R[s][policy[s]] for s in S], dtype=float)
V = np.linalg.solve(np.eye(len(S)) - gamma * Pmat, Rvec)

print("Action space :", A)
print("Discount gamma:", gamma)
print("Transition matrix under policy:\n", Pmat)
print("Reward vector under policy:", Rvec)
for s, v in zip(S, V):
   print(f"V({s}) = {v:.3f}")
```

Sample Output:

```
Action space : ['Walk', 'Shop']
Discount gamma: 0.9
Transition matrix under policy:
 [[0.8 0.2]
 [0.5 0.5]]
Reward vector under policy: [5. 3.]
V(Sunny) = 45.068
V(Rainy) = 42.329
```

Challenge 3: Policy, Value Functions and the Bellman Equation

Problem Statement: On a four-by-four grid world with terminal corners, evaluate the uniform-random policy using iterative policy evaluation based on the Bellman expectation equation, then derive action-values for one state to identify the greedy action.

Objectives: Implement iterative policy evaluation for the state-value function, compute the action-value function from it, and select the best action for a chosen state.

Input Format: No external input; the grid size, terminal states and per-step reward are fixed in the program.

Output Format: The state-value grid V(s), the action-values Q(s,a) for a chosen state, and the greedy action derived from them.

Constraints: A four-by-four grid; reward of minus one per step; terminals at cells 0 and 15; convergence threshold of 1e-4.

Evaluation Criteria: Correct Bellman updates, demonstrated convergence, and a correct action-value and greedy-action derivation.

Reference Solution (Python):

```
# Challenge 3: Policy & value functions via the Bellman equation.
# 4x4 grid world, terminal corners, reward -1 per step, uniform-random policy.
# Iterative policy evaluation gives the state-value V(s) (Bellman expectation).
# We also derive an action-value Q(s,a) for one state.
import numpy as np
SIZE, N = 4, 16
TERM = {0, 15}
ACTIONS = {0:(-1,0), 1:(1,0), 2:(0,-1), 3:(0,1)}  # U D L R

def nxt(s, a):
   if s in TERM: return s
```

```
    r, c = divmod(s, SIZE); dr, dc = ACTIONS[a]
    r = min(SIZE-1, max(0, r+dr)); c = min(SIZE-1, max(0, c+dc))
    return r*SIZE + c

def evaluate(gamma=1.0, theta=1e-4):
    V = np.zeros(N)
    while True:
        delta = 0
        for s in range(N):
            if s in TERM: continue
            v = V[s]
            V[s] = sum(0.25 * (-1 + gamma * V[nxt(s, a)]) for a in ACTIONS)
            delta = max(delta, abs(v - V[s]))
        if delta < theta: break
    return V

V = evaluate()
print("State-value V(s) under uniform-random policy (Bellman expectation):")
print(np.round(V.reshape(SIZE, SIZE), 2))
s = 5
print(f"\nAction-value Q(s={s}, a) = -1 + V(next):")
for a, name in zip(ACTIONS, ["Up","Down","Left","Right"]):
    print(f"  {name:<5}: {-1 + V[nxt(s, a)]:.2f}")
print("Best action from Bellman optimality:", ["Up","Down","Left","Right"]
[int(np.argmax([-1+V[nxt(s,a)] for a in ACTIONS]))])
```

Sample Output:

```
State-value V(s) under uniform-random policy (Bellman expectation):
[[  0. -14. -20. -22.]
 [-14. -18. -20. -20.]
 [-20. -20. -18. -14.]
 [-22. -20. -14.   0.]]

Action-value Q(s=5, a) = -1 + V(next):
  Up   : -15.00
  Down : -21.00
  Left : -15.00
  Right: -21.00
Best action from Bellman optimality: Up
```

Challenge 4: Exploration-Exploitation Trade-off: Epsilon-Greedy versus Softmax

Problem Statement: On a ten-armed bandit, implement epsilon-greedy and softmax action selection and compare how their parameters balance exploration against exploitation.

Objectives: Implement both selection strategies, track incremental value estimates, and report the average reward and the arm chosen for a range of parameter values.

Input Format: No external input; the true arm values are drawn from a fixed-seed normal distribution.

Output Format: The optimal arm, and for each strategy the average reward and selected arm across several parameter settings.

Constraints: Ten arms and two thousand steps; epsilon in {0, 0.1, 0.3}; temperature tau in {0.1, 0.5, 1.0}.

Evaluation Criteria: Correct implementation of both selection rules and a meaningful comparison of their learning efficiency.

Reference Solution (Python):

```
# Challenge 4: Exploration-exploitation trade-off.
# 10-armed bandit. Compare epsilon-greedy and softmax action selection.
```

```
import numpy as np
np.random.seed(1)
K, STEPS = 10, 2000
q_true = np.random.normal(0, 1, K)

def eps_greedy(eps):
    Q = np.zeros(K); Nc = np.zeros(K); total = 0
    for _ in range(STEPS):
        a = np.random.randint(K) if np.random.random() < eps else int(np.argmax(Q))
        r = np.random.normal(q_true[a], 1); Nc[a]+=1; Q[a]+=(r-Q[a])/Nc[a]; total+=r
    return total/STEPS, int(np.argmax(Q))

def softmax(tau):
    Q = np.zeros(K); Nc = np.zeros(K); total = 0
    for _ in range(STEPS):
        p = np.exp(Q/tau); p/=p.sum()
        a = np.random.choice(K, p=p)
        r = np.random.normal(q_true[a], 1); Nc[a]+=1; Q[a]+=(r-Q[a])/Nc[a]; total+=r
    return total/STEPS, int(np.argmax(Q))

print(f"Optimal arm = {int(np.argmax(q_true))}  (true value {q_true.max():.2f})\n")
print("epsilon-greedy:")
for e in [0.0, 0.1, 0.3]:
    avg, best = eps_greedy(e); print(f"  eps={e}: avg reward {avg:.3f}, arm chosen {best}")
print("softmax:")
for t in [0.1, 0.5, 1.0]:
    avg, best = softmax(t); print(f"  tau={t}: avg reward {avg:.3f}, arm chosen {best}")
```

Sample Output:

```
Optimal arm = 6  (true value 1.74)

epsilon-greedy:
  eps=0.0: avg reward 1.621, arm chosen 0
  eps=0.1: avg reward 1.447, arm chosen 0
  eps=0.3: avg reward 1.172, arm chosen 6
softmax:
  tau=0.1: avg reward 0.838, arm chosen 4
  tau=0.5: avg reward 1.483, arm chosen 6
  tau=1.0: avg reward 1.085, arm chosen 6
```

Challenge 5: Reward Shaping to Accelerate Learning

Problem Statement: Show that potential-based reward shaping accelerates learning on a sparse-reward corridor without changing the optimal policy, by comparing a sparse-reward agent against a shaped-reward agent.

Objectives: Implement Q-learning with and without potential-based shaping and measure how quickly each agent reaches the goal.

Input Format: No external input; the corridor length, goal position and potential function are fixed in code.

Output Format: For each variant, the number of episodes out of four hundred that reached the goal and the first successful episode.

Constraints: Corridor length twelve; potential function phi(s) = minus (goal minus s); discount 0.99; exploration rate 0.2.

Evaluation Criteria: A correct shaping term and clear evidence that shaping speeds up learning while preserving the optimal policy.

Reference Solution (Python):

```
# Challenge 5: Reward shaping accelerates learning and prevents poor policies.
# 1D corridor length 12, goal at the end, sparse reward (+1 only at goal).
# Potential-based shaping adds gamma*phi(s')-phi(s) using distance-to-goal.
# We compare how many episodes reach the goal (out of 400) and the first hit.
import numpy as np
np.random.seed(0)
N, GOAL, gamma = 12, 11, 0.99

def phi(s): return -(GOAL - s)          # closer to goal -> higher potential

def train(shaped):
    Q = np.zeros((N, 2)); eps = 0.2
    successes, first = 0, None
    for ep in range(400):
        s = 0
        for t in range(200):
            a = np.random.randint(2) if np.random.random() < eps else int(np.argmax(Q[s]))
            s2 = min(GOAL, s+1) if a == 1 else max(0, s-1)
            base = 1.0 if s2 == GOAL else 0.0
            r = base + (gamma*phi(s2) - phi(s) if shaped else 0.0)
            Q[s, a] += 0.1 * (r + gamma*np.max(Q[s2]) - Q[s, a])
            s = s2
            if s == GOAL:
                successes += 1
                if first is None: first = ep
                break
    return successes, first

for shaped, name in [(False, "Sparse reward only"), (True, "With reward shaping")]:
    succ, first = train(shaped)
    hit = f"first reached at episode {first}" if first is not None else "goal never reached"
    print(f"{name:<20}: {succ:>3}/400 episodes reached goal | {hit}")
print("\nShaping supplies a dense learning signal, so the agent solves the task")
print("almost immediately, while sparse reward leaves it wandering for long.")
```

Sample Output:

```
Sparse reward only  :   0/400 episodes reached goal | goal never reached
With reward shaping : 400/400 episodes reached goal | first reached at episode 0

Shaping supplies a dense learning signal, so the agent solves the task
almost immediately, while sparse reward leaves it wandering for long.
```

Challenge 6: Challenges of Reinforcement Learning in Robotics

Problem Statement: Train a Q-learning agent on a five-by-five grid containing hazard cells, then test the learned policy under increasing transition noise and different training budgets. Quantify the success rate and unsafe-cell entries to illustrate the safety, sample-efficiency and environment-variability challenges of real-world robotics.

Objectives: Train under a safety penalty, evaluate robustness to environment noise, and compare the effect of the training budget on performance.

Input Format: No external input; the grid, hazard cells, goal and noise levels are fixed in the program.

Output Format: For each training budget and noise level, the success rate and the unsafe-entry rate.

Constraints: A five-by-five grid; hazards at cells 12 and 17; slip noise in {0, 0.1, 0.3}; training budgets of 200 and 2000 episodes.

Evaluation Criteria: Correct handling of hazard cells and a clear demonstration of how noise and sample budget affect agent performance.

Reference Solution (Python):

```
# Challenge 6: Challenges of RL in robotics - safety, sample efficiency,
# and environment variability. We train Q-learning on a grid, then test the
# learned policy under increasing transition noise (slip probability) and
# count unsafe-cell entries, showing how variability degrades performance.
import numpy as np
np.random.seed(0)
SIZE, N = 5, 25
GOAL, HAZARD = 24, {12, 17}          # hazard = unsafe cells (safety concern)
ACT = {0:(-1,0),1:(1,0),2:(0,-1),3:(0,1)}

def move(s, a):
    r, c = divmod(s, SIZE); dr, dc = ACT[a]
    r = min(SIZE-1, max(0, r+dr)); c = min(SIZE-1, max(0, c+dc))
    return r*SIZE + c

def reward(s):
    if s == GOAL: return 20, True
    if s in HAZARD: return -20, False
    return -1, False

def train(episodes):
    Q = np.zeros((N, 4)); eps = 0.2
    for _ in range(episodes):
        s = 0
        for _ in range(60):
            a = np.random.randint(4) if np.random.random() < eps else int(np.argmax(Q[s]))
            s2 = move(s, a); r, done = reward(s2)
            Q[s, a] += 0.1 * (r + 0.95*np.max(Q[s2]) - Q[s, a]); s = s2
            if done or s in HAZARD: break
    return Q

def test(Q, slip):
    succ = viol = 0
    for _ in range(500):
        s = 0
        for _ in range(60):
            a = int(np.argmax(Q[s]))
            if np.random.random() < slip: a = np.random.randint(4)   # variability
            s = move(s, a)
            if s in HAZARD: viol += 1; break
            if s == GOAL: succ += 1; break
    return succ/500, viol/500

for samples in [200, 2000]:
    Q = train(samples)
    print(f"\nTrained on {samples} episodes (sample efficiency):")
    for slip in [0.0, 0.1, 0.3]:
        sr, vr = test(Q, slip)
        print(f"  noise={slip:>3}:  success {sr:5.1%} | unsafe-entry rate {vr:5.1%}")
print("\nMore samples improve the policy; higher environment noise lowers success")
print("and raises unsafe entries - the core robotics challenges.")
```

Sample Output:

```
Trained on 200 episodes (sample efficiency):
  noise=0.0:  success 100.0% | unsafe-entry rate  0.0%
  noise=0.1:  success 97.6% | unsafe-entry rate  2.4%
  noise=0.3:  success 86.8% | unsafe-entry rate 13.2%
```

```
Trained on 2000 episodes (sample efficiency):
  noise=0.0:  success 100.0% | unsafe-entry rate  0.0%
  noise=0.1:  success 92.6% | unsafe-entry rate  7.4%
  noise=0.3:  success 81.8% | unsafe-entry rate 18.2%

More samples improve the policy; higher environment noise lowers success
and raises unsafe entries - the core robotics challenges.
```

Challenge 7: Influence of the Discount Factor on Decision-Making

Problem Statement: Demonstrate how the discount factor changes an agent's preference between a small immediate reward and a larger delayed reward, and how it scales the return of an episodic reward sequence.

Objectives: Compute the discounted value of a greedy-now action versus a patient action across several discount factors, and compute the return of a fixed reward sequence for several discount factors.

Input Format: No external input; the reward magnitudes and their delays are fixed in code.

Output Format: A table of greedy versus patient values with the optimal choice for each discount factor, plus the discounted return of a sample reward sequence.

Constraints: Immediate reward of plus two after one step; delayed reward of plus ten after four steps; discount factor in {0.1, 0.5, 0.9, 0.99}.

Evaluation Criteria: Correct discounting and a correct account of how the discount factor shifts short-term versus long-term preference.

Reference Solution (Python):

```
# Challenge 7: Influence of the discount factor (gamma) on short- vs
# long-term decisions. A 2-action choice: a small immediate reward now, or
# a path that yields a large reward later. Value iteration for several gamma.
import numpy as np
# States: 0=start, 1..3 = path to big reward, 4=terminal-big, 5=terminal-small
# Action A ("greedy now"): start->small terminal, reward +2 immediately.
# Action B ("patient")   : start->1->2->3->big terminal, reward +10 at the end.
def value_of_patient(gamma):   # +10 received after 4 steps
    return gamma**4 * 10
def value_of_greedy(gamma):    # +2 received after 1 step
    return gamma * 2

print("gamma | greedy-now value | patient value | optimal choice")
for g in [0.1, 0.5, 0.9, 0.99]:
    vg, vp = value_of_greedy(g), value_of_patient(g)
    choice = "Patient (long-term)" if vp > vg else "Greedy (short-term)"
    print(f" {g:<4} |      {vg:6.3f}      |   {vp:6.3f}     | {choice}")

print("\nReturn of a fixed reward sequence [1,1,1,1,1] under different gamma (episodic):")
rews = [1,1,1,1,1]
for g in [0.1, 0.5, 0.9, 1.0]:
    G = sum((g**t)*r for t, r in enumerate(rews))
    print(f"  gamma={g}: return = {G:.3f}")
print("Low gamma favours immediate reward; high gamma values long-term outcomes.")
```

Sample Output:

```
gamma | greedy-now value | patient value | optimal choice
 0.1  |       0.200      |    0.001     | Greedy (short-term)
 0.5  |       1.000      |    0.625     | Greedy (short-term)
 0.9  |       1.800      |    6.561     | Patient (long-term)
 0.99 |       1.980      |    9.606     | Patient (long-term)
```

```
Return of a fixed reward sequence [1,1,1,1,1] under different gamma (episodic):
  gamma=0.1: return = 1.111
  gamma=0.5: return = 1.938
  gamma=0.9: return = 4.095
  gamma=1.0: return = 5.000
Low gamma favours immediate reward; high gamma values long-term outcomes.
```

Challenge 8: Model-Free versus Model-Based Reinforcement Learning

Problem Statement: On the same four-by-four grid, compare model-based value iteration, which uses the known model, with model-free Q-learning, which learns from sampled interaction. Report the greedy path each produces and the number of environment samples each consumes.

Objectives: Implement both methods and compare the resulting policies and their sample cost.

Input Format: No external input; the grid, goal and rewards are fixed in the program.

Output Format: The number of samples used and the greedy start-to-goal path for each method.

Constraints: A four-by-four grid; discount 0.9; Q-learning run for six hundred episodes with exploration rate 0.2.

Evaluation Criteria: Correct implementation of both methods and an accurate comparison of their strengths and limitations.

Reference Solution (Python):

```
# Challenge 8: Model-free vs model-based RL in a dynamic environment.
# Same 4x4 grid. Model-based value iteration uses the known model (no sampling).
# Model-free Q-learning learns purely from sampled interaction. We compare the
# greedy path each produces from the start and how many samples each needed.
import numpy as np
np.random.seed(0)
SIZE, N = 4, 16; GOAL = 15
ACT = {0:(-1,0),1:(1,0),2:(0,-1),3:(0,1)}; NAMES = ["U","D","L","R"]
def move(s, a):
    r, c = divmod(s, SIZE); dr, dc = ACT[a]
    r = min(SIZE-1, max(0, r+dr)); c = min(SIZE-1, max(0, c+dc)); return r*SIZE+c
def rew(s): return (10, True) if s == GOAL else (-1, False)

# --- Model-based: value iteration (uses the model directly) ---
V = np.zeros(N)
for _ in range(100):
    for s in range(N):
        if s == GOAL: continue
        V[s] = max(rew(move(s,a))[0] + 0.9*V[move(s,a)] for a in ACT)
pol_mb = [int(np.argmax([rew(move(s,a))[0] + 0.9*V[move(s,a)] for a in ACT])) for s in
range(N)]

# --- Model-free: Q-learning (samples the environment) ---
Q = np.zeros((N,4)); samples = 0; eps = 0.2
for ep in range(600):
    s = 0
    for _ in range(50):
        a = np.random.randint(4) if np.random.random()<eps else int(np.argmax(Q[s]))
        s2 = move(s,a); r, done = rew(s2); samples += 1
        Q[s,a] += 0.1*(r + 0.9*np.max(Q[s2]) - Q[s,a]); s = s2
        if done: break
pol_mf = [int(np.argmax(Q[s])) for s in range(N)]
```

```
def path(pol):                    # greedy walk from start state 0
    s, steps = 0, []
    for _ in range(30):
        a = pol[s]; steps.append(NAMES[a]); s = move(s, a)
        if s == GOAL: break
    return steps

pmb, pmf = path(pol_mb), path(pol_mf)
print("Model-based (value iteration): samples used = 0 (uses known model)")
print(f"   greedy path from start: {'->'.join(pmb)}  ({len(pmb)} steps to goal)")
print(f"Model-free (Q-learning)     : samples used = {samples}")
print(f"   greedy path from start: {'->'.join(pmf)}  ({len(pmf)} steps to goal)")
print("\nBoth reach the goal optimally. Model-based needs the model but no samples;")
print("model-free needs many samples yet adapts when the model is unknown/changing.")
```

Sample Output:

```
Model-based (value iteration): samples used = 0 (uses known model)
   greedy path from start: D->D->D->R->R->R  (6 steps to goal)
Model-free (Q-learning)     : samples used = 4677
   greedy path from start: D->R->D->R->D->R  (6 steps to goal)

Both reach the goal optimally. Model-based needs the model but no samples;
model-free needs many samples yet adapts when the model is unknown/changing.
```

Challenge 9: Q-Learning Update and Convergence Behaviour

Problem Statement: Demonstrate the Q-learning action-value update with one worked calculation, then train on a six-state corridor and track the largest Q-value change per block of episodes to show convergence.

Objectives: Show a single update step in full, train the agent to convergence, and display the final Q-table.

Input Format: No external input; the corridor length, learning rate, discount factor and exploration rate are fixed in code.

Output Format: The worked update, the maximum Q-value change per fifty-episode block, and the final Q-table.

Constraints: Six states; learning rate 0.5; discount 0.9; exploration rate 0.2.

Evaluation Criteria: A correct application of the Q-learning update rule and clear evidence of convergence through diminishing changes.

Reference Solution (Python):

```
# Challenge 9: Q-learning update of the action-value function and convergence.
# Small 1x6 corridor, goal at the end. We show one worked update, then track
# the largest Q change per block of episodes to demonstrate convergence.
import numpy as np
np.random.seed(0)
N, GOAL = 6, 5
def move(s,a): return (min(GOAL,s+1) if a==1 else max(0,s-1))
def rew(s): return (10,True) if s==GOAL else (-1,False)
Q = np.zeros((N,2)); alpha, gamma, eps = 0.5, 0.9, 0.2

# one worked update from state 4 taking action 'right'
s, a = 4, 1; s2 = move(s,a); r,_ = rew(s2)
old = Q[s,a]; Q[s,a] += alpha*(r + gamma*np.max(Q[s2]) - Q[s,a])
print(f"Worked update: Q({s},right) = {old:.2f} + {alpha}*({r} + {gamma}
*{np.max(Q[s2]):.2f} - {old:.2f}) = {Q[s,a]:.2f}\n")

Q = np.zeros((N,2))
print("Convergence (max |Q change| per 50 episodes):")
```

```
for block in range(6):
    maxd = 0
    for _ in range(50):
        s = 0
        for _ in range(30):
            a = np.random.randint(2) if np.random.random()<eps else int(np.argmax(Q[s]))
            s2 = move(s,a); r, done = rew(s2)
            d = alpha*(r + gamma*np.max(Q[s2]) - Q[s,a]); Q[s,a]+=d
            maxd = max(maxd, abs(d)); s = s2
            if done: break
    print(f"  episodes {block*50+1:>3}-{block*50+50}: max change = {maxd:.4f}")
print("\nFinal Q-table (rows=state 0..5, cols=[left,right]):")
print(np.round(Q,2))
```

Sample Output:

```
Worked update: Q(4,right) = 0.00 + 0.5*(10 + 0.9*0.00 - 0.00) = 5.00

Convergence (max |Q change| per 50 episodes):
  episodes   1-50: max change = 5.0000
  episodes  51-100: max change = 1.0946
  episodes 101-150: max change = 0.0138
  episodes 151-200: max change = 0.0002
  episodes 201-250: max change = 0.0000
  episodes 251-300: max change = 0.0000

Final Q-table (rows=state 0..5, cols=[left,right]):
[[ 1.81  3.12]
 [ 1.81  4.58]
 [ 3.12  6.2 ]
 [ 4.58  8.  ]
 [ 6.2  10.  ]
 [ 0.    0.  ]]
```

Challenge 10: On-Policy versus Off-Policy Learning: SARSA and Q-Learning

Problem Statement: On a cliff-walking grid, implement SARSA (on-policy) and Q-learning (off-policy) and compare the greedy paths they learn, to highlight the difference between their update targets.

Objectives: Implement both control algorithms and evaluate the greedy-path return of each.

Input Format: No external input; the grid, cliff cells, start and goal positions are fixed in the program.

Output Format: The greedy-path return for SARSA and for Q-learning, with a short explanation of the difference.

Constraints: A four-by-twelve grid; a cliff along the bottom row; learning rate 0.5; discount 1.0; exploration rate 0.1; five hundred episodes.

Evaluation Criteria: Correct on-policy and off-policy update targets and a correct account of their practical trade-offs.

Reference Solution (Python):

```
# Challenge 10: On-policy (SARSA) vs off-policy (Q-learning).
# Cliff-walking style grid: a bottom-row "cliff" gives -100 and resets.
# Q-learning learns the optimal (risky, cliff-edge) path; SARSA learns a
# safer path because it accounts for exploratory moves. We compare returns.
import numpy as np
np.random.seed(0)
ROWS, COLS = 4, 12
START, GOAL = (3,0), (3,11)
CLIFF = {(3,c) for c in range(1,11)}
```

```
ACT = {0:(-1,0),1:(1,0),2:(0,-1),3:(0,1)}

def step(s, a):
    r, c = s; dr, dc = ACT[a]
    r = min(ROWS-1, max(0, r+dr)); c = min(COLS-1, max(0, c+dc)); ns = (r,c)
    if ns in CLIFF: return START, -100, False
    if ns == GOAL:  return ns, 0, True
    return ns, -1, False

def egreedy(Q, s, eps):
    return np.random.randint(4) if np.random.random()<eps else int(np.argmax(Q[s]))

def train(method, episodes=500, alpha=0.5, gamma=1.0, eps=0.1):
    Q = np.zeros((ROWS,COLS,4))
    for _ in range(episodes):
        s = START; a = egreedy(Q, s, eps)
        for _ in range(200):
            s2, r, done = step(s, a); a2 = egreedy(Q, s2, eps)
            if method == "sarsa":
                target = r + gamma*Q[s2][a2]
            else:  # q-learning
                target = r + gamma*np.max(Q[s2])
            Q[s][a] += alpha*(target - Q[s][a]); s, a = s2, a2
            if done: break
    return Q

def greedy_return(Q):
    s = START; total = 0
    for _ in range(200):
        a = int(np.argmax(Q[s])); s, r, done = step(s, a); total += r
        if done: break
    return total

for m in ["sarsa", "q-learning"]:
    Q = train(m)
    print(f"{m:<11}: greedy-path return = {greedy_return(Q)}")
print("\nQ-learning (off-policy) finds the optimal cliff-edge path (higher return");
print("but risky). SARSA (on-policy) prefers a safer path away from the cliff.")
```

Sample Output:

```
sarsa      : greedy-path return = -16
q-learning : greedy-path return = -12

Q-learning (off-policy) finds the optimal cliff-edge path (higher return
but risky). SARSA (on-policy) prefers a safer path away from the cliff.
```

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and	Applies concepts with	Limited problem-solving ability;	Unable to solve or

		applies concepts effectively in real scenarios	moderate accuracy	requires guidance	apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately